

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ORGANIZACIÓN DE LENGUAJES Y
COMPILADORES 1
PRIMER SEMESTRE 2026
PRACTICA 2
Andy Jefferson González Fuentes 202231338



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

MANUAL DE USUARIO

Wison IDE es una herramienta web que permite definir gramáticas libres de contexto usando el lenguaje Wison, compilarlas en analizadores sintácticos LL(1) y probar cadenas de entrada contra esos analizadores. La herramienta detecta errores léxicos, sintácticos y semánticos, y genera el árbol de derivación para las cadenas aceptadas.

La aplicación tiene dos secciones principales:

- **Editor:** donde se escribe o carga el código Wison y se compila el analizador.
- **Tester:** donde se selecciona un analizador y se evalúan cadenas de entrada.

Para ejecutar la aplicación se necesitan dos terminales abiertas simultáneamente:
Backend:

```
cd wison-project/backend  
npm run build:parser  
npm run dev
```

El servidor quedará disponible en <http://localhost:3000>

Fronted:

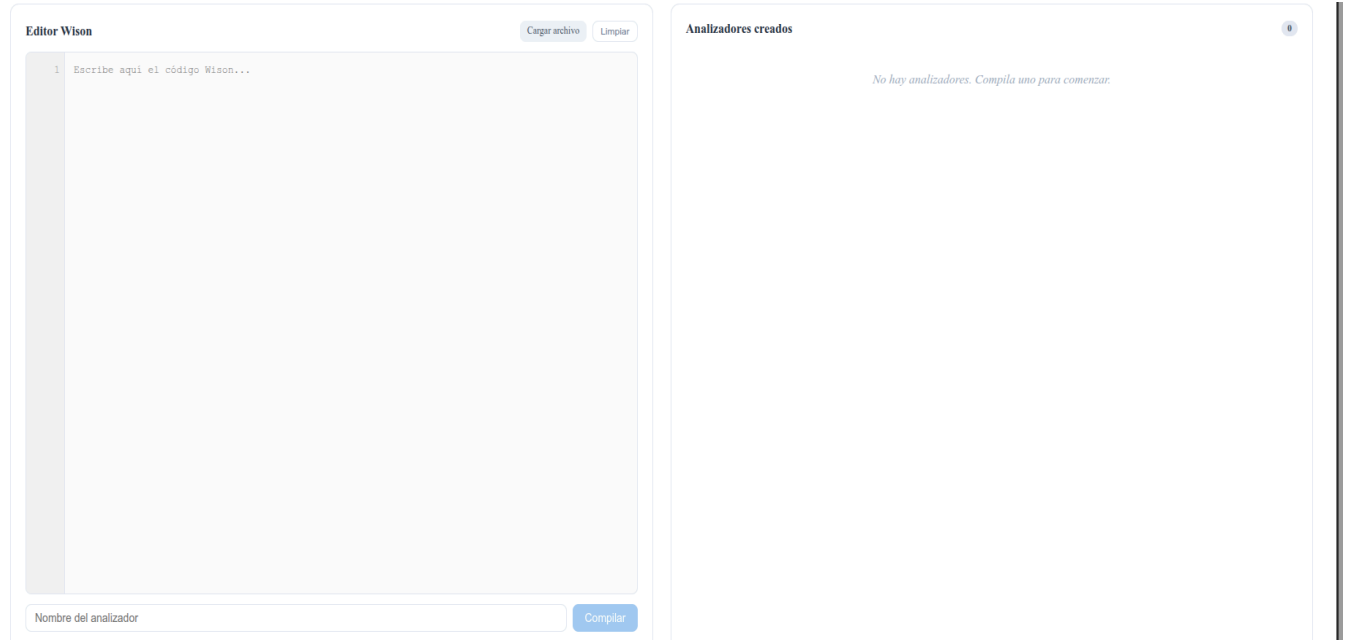
```
cd wison-project/frontend  
ng serve
```

La aplicación quedará disponible en <http://localhost:4200>

Sección editor: La sección Editor está dividida en dos paneles: el editor de código a la izquierda y la lista de analizadores creados a la derecha.

Hay dos formas de ingresar el código Wison:

1. **Escribir directamente:** hacer clic en el área de texto del editor y escribir el código. El panel muestra números de línea para facilitar la identificación de errores.
2. **Cargar desde archivo:** hacer clic en el botón "Cargar archivo" y seleccionar un archivo .wison desde el equipo. El nombre del archivo se sugiere automáticamente como nombre del analizador.



Compilar un analizador

1. Escribir o cargar el código Wison en el editor.
2. Escribir un nombre para el analizador en el campo de texto junto al botón Compilar.
3. Hacer clic en el botón Compilar.

Panel de errores: El panel de errores muestra todos los problemas encontrados durante la compilación organizados por tipo:

Tipo	Color	Descripción
Léxico	Morado	Carácter no reconocido por el lenguaje Wison. Ejemplo: @ o %
Sintáctico	Rojo	Estructura incorrecta. Ejemplo: falta <- o falta ;
Semántico	Naranja	Problema lógico. Ejemplo: NT no declarado, recursión izquierda, conflicto en tabla LL(1)
Warning	Amarillo	Advertencia no bloqueante. Ejemplo: NT declarado pero sin producción

El sistema puede mostrar múltiples errores simultáneamente gracias a la recuperación de errores implementada. Esto permite corregir varios problemas de una sola vez en lugar de corregir uno por uno.

Lista de analizadores creados: El panel derecho muestra todos los analizadores compilados exitosamente. Para cada analizador se muestra:

- Nombre del analizador
- Cantidad de terminales y no terminales
- Símbolo inicial
- Advertencias si las hay

Para eliminar un analizador, hacer clic en el botón Eliminar junto a su nombre. Los analizadores se guardan automáticamente en disco y persisten entre reinicios del servidor.

Para limpiar el editor sin eliminar analizadores, hacer clic en el botón Limpiar.

Analizadores creados

7

expresiones 6 terminales · 6 no terminales · inicial: %_Inicio	Eliminar
operadores 9 terminales · 7 no terminales · inicial: %_Inicio	Eliminar
variables 19 terminales · 9 no terminales · inicial: %_Programa	Eliminar
if-anidado 10 terminales · 8 no terminales · inicial: %_Programa	Eliminar
listas 11 terminales · 10 no terminales · inicial: %_Prog	Eliminar
matematica 11 terminales · 7 no terminales · inicial: %_Programa	Eliminar
complejo 17 terminales · 8 no terminales · inicial: %_Programa ⚠ 2 advertencia(s)	Eliminar

Sección de tester: La sección Tester permite evaluar cadenas de entrada contra un analizador compilado. Está dividida en el panel de evaluación a la izquierda y el árbol de derivación a la derecha.

Evaluar cadena:

1. Seleccionar un analizador del menú desplegable.
2. Escribir la cadena de entrada en el campo de texto.
3. Hacer clic en Evaluar o presionar la tecla **Enter**.

El resultado aparece inmediatamente:

- **Cadena ACEPTADA (verde):** la cadena pertenece al lenguaje definido por la gramática. Se muestra el árbol de derivación.

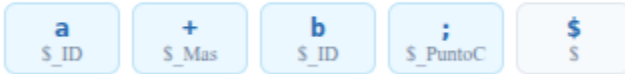
Cadena ACEPTADA

- **Cadena RECHAZADA (rojo):** la cadena no pertenece al lenguaje. Se muestran los errores con detalle de qué token se esperaba y cuál se encontró.

Cadena RECHAZADA

Tokens reconocidos: Debajo del resultado aparece la lista de tokens que el analizador identificó en la cadena. Cada token muestra su lexema (el texto real) y su nombre de terminal. El token \$ indica el fin de la cadena.

TOKENS RECONOCIDOS



Árbol de derivación: Cuando una cadena es aceptada, el panel derecho muestra el árbol de derivación gráfico con nodos de tres colores:

Color del nodo	Tipo	Descripción
Azul	No terminal	Nodo interno del árbol. Representa una variable de la gramática como %_Expr o %_Term.
Verde	Terminal	Hoja del árbol. Muestra el lexema real entre comillas, por ejemplo "id" o "+".
Morado punteado	Épsilon (ϵ)	Hoja que representa la cadena vacía. Aparece cuando una producción deriva a ϵ .

Probar analizador

Analizador:

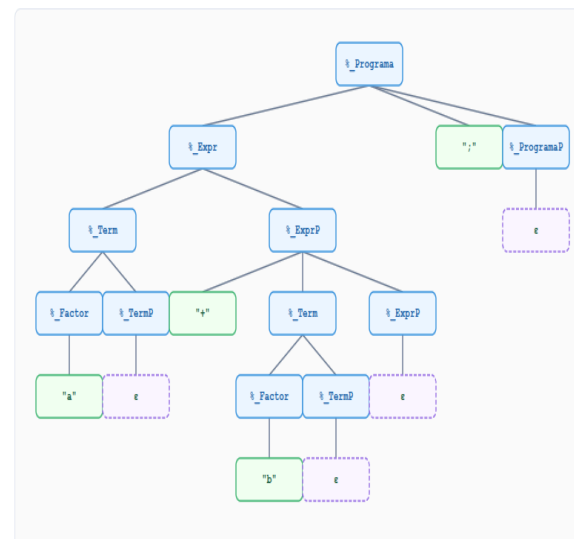
Programa de entrada:

Cadena ACEPTADA

TOKENS RECONOCIDOS

\$_ID \$_Mas \$_ID \$_PuntoC \$

Árbol de derivación



Estructura del Lenguaje wison

Bloque lex terminales: Define los símbolos terminales de la gramática usando expresiones regulares:

```
Terminal $_Nombre <- expresión ;
```

Expresión	Descripción	Ejemplo
'texto'	Literal exacto entre comillas simples	Terminal \$_Mas <- '+' ;
[aA-zZ]	Rango alfabético completo	Terminal \$_Letra <- [aA-zZ] ;
[0-9]	Rango de dígitos 0 al 9	Terminal \$_Dig <- [0-9] ;
expresión*	Cero o más repeticiones (Kleene)	Terminal \$_Digs <- [0-9]* ;
expresión+	Una o más repeticiones	Terminal \$_Digs <- [0-9]+ ;
expresión?	Cero o una vez (opcional)	Terminal \$_Dig <- [0-9]? ;
(expr1)(expr2)	Concatenación de expresiones	Terminal \$_D <- ([0-9]+)(\$_Punto)([0-9]+) ;

Bloque syntax producciones: Define los no terminales, el símbolo inicial y las producciones de la gramática:

```

No_Terminal %_Nombre ;           # declaración de NT
Initial_Sim %_Nombre ;           # símbolo inicial (obligatorio)
%_A <= %_B $_C | %_D | ;        # producción con alternativas y
épsilon

```

Elemento	Descripción
No_Terminal %_Nombre ;	Declara un no terminal. Debe declararse antes de usarlo en producciones.
Initial_Sim %_Nombre ;	Indica cuál es el símbolo inicial. Obligatorio, solo uno por gramática.
%_A <= %_B \$_C ;	Producción: %_A deriva a la secuencia %_B \$_C.
%_D	Alternativa adicional para la misma producción.
;	Alternativa épsilon — el NT puede derivar la cadena vacía.

Errores desde Tester

ERRORES (3)

Sintáctico Línea 1, Col 6

Error sintáctico en "\$" (""). No hay producción para M["%_TermP"]["\$"]. Tokens válidos: \$_Por, \$_Div, \$_Mas, \$_Menos, \$_PuntoC, \$_P_Ce

Esperaba: \$_Por, \$_Div, \$_Mas, \$_Menos, \$_PuntoC, \$_P_Ce

Encontró: \$

Sintáctico Línea 1, Col 6

Error sintáctico en "\$" (""). No hay producción para M["%_ExprP"]["\$"]. Tokens válidos: \$_Mas, \$_Menos, \$_PuntoC, \$_P_Ce

Esperaba: \$_Mas, \$_Menos, \$_PuntoC, \$_P_Ce

Encontró: \$

Errores desde Editor

ERRORES (3)

Léxico Línea 1, Col 0

Carácter no reconocido: "@" (U+0040)

Sintáctico Línea 5, Col 24

Error en declaración de Terminal en línea 5. Formato esperado: Terminal \$_Nombre <- expresión ;

Léxico Línea 7, Col 0

Carácter(es) no reconocido(s): "wison"

Árboles:

